arXiv:2510.08562v1 [cs.CV] 9 Oct 2025

ResAD: Normalized Residual Trajectory Modeling for End-to-End Autonomous Driving

Zhiyu Zheng[1,◇] Shaoyu Chen[2,†] Haoran Yin[2] Xinbang Zhang[2] Jialv Zou[3,◇]
Xinggang Wang[3] Qian Zhang[2] Lefei Zhang[1, ✉]

[1] School of Computer Science, Wuhan University [2] Horizon Robotics
[3] School of EIC, Huazhong University of Science & Technology
https://duckyee728.github.io/ResAD

Abstract

*End-to-end autonomous driving (E2EAD) systems, which learn to predict future trajectories directly from sensor data, are fundamentally challenged by the inherent spatio-temporal imbalance of trajectory data. This imbalance creates a significant optimization burden, causing models to learn spurious correlations instead of causal inference, while also prioritizing uncertain, distant predictions, thereby compromising immediate safety. To address these issues, we propose **ResAD**, a novel **Normalized Residual Trajectory Modeling** framework. Instead of predicting the future trajectory directly, our approach reframes the learning task to predict the **residual** deviation from a deterministic **inertial reference**. The inertial reference serves as a counterfactual, forcing the model to move beyond simple pattern recognition and instead identify the underlying causal factors (e.g., traffic rules, obstacles) that necessitate deviations from a default, inertially-guided path. To deal with the optimization imbalance caused by uncertain, long-term horizons, **ResAD** further incorporates Point-wise Normalization of the predicted residual. It re-weights the optimization objective, preventing large-magnitude errors associated with distant, uncertain waypoints from dominating the learning signal. Extensive experiments validate the effectiveness of our framework. On the NAVSIM benchmark, **ResAD** achieves a state-of-the-art PDMS of 88.6 using a vanilla diffusion policy with only two denoising steps, demonstrating that our approach significantly simplifies the learning task and improves model performance. The code will be released to facilitate further research.*

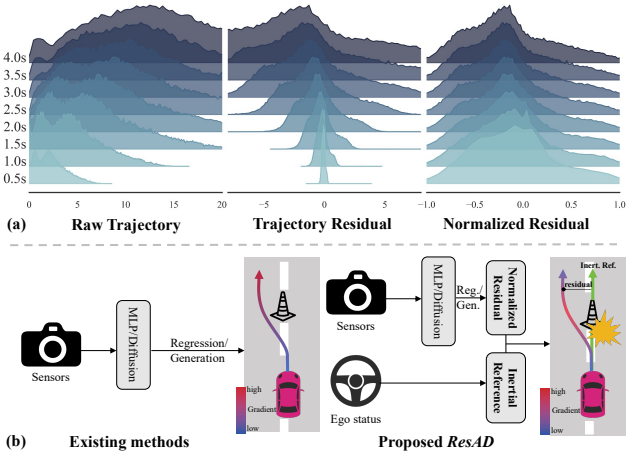

Figure 1. **(a)** Comparison of trajectory distributions under different modeling strategies. Raw trajectories exhibit significant mean drift and increasing variance over the prediction horizon. **Trajectory Residual Modeling** centers the distribution around zero. **Point-wise Residual Normalization** further stabilizes variance for a simpler learning objective. **(b)** Comparisons between existing methods and the proposed ***ResAD***. Instead of predicting the trajectory directly, ***ResAD*** obtains an inertial reference (green arrow) as a counterfactual baseline. This forces the model to learn **not what to do**, but **why it must deviate from this baseline**, effectively linking actions (*i.e.*, residuals) to their causal sources, like obstacles, rather than to statistical correlations.

1. Introduction

Conventional autonomous driving systems rely on a modular pipeline of perception, prediction, and planning components [7, 26, 29]. This cascaded design is prone to error propagation, leading to suboptimal or unsafe driving. In response to these limitations, End-to-End Autonomous Driving (E2EAD) has emerged as a compelling alternative [2, 12]. E2EAD reframes the driving problem by learning a direct mapping from raw sensor inputs to a future driv-

◇ Intern of Horizon Robotics.
† Project lead. ✉ Corresponding author.

ing trajectory, from which control commands are derived, all within a single, unified framework [4, 10, 15, 33].

Recent years have witnessed extensive research into E2EAD methods, focusing on developing more effective representations [3, 17, 32], enhancing sensor fusion techniques [5, 13, 28, 37], and designing advanced architectures [16, 31, 35]. However, these existing methods are all attempting to answer the same question: "***What is the future trajectory?***" We argue this approach is inherently limited. As shown in Fig.1(a), the raw trajectory data exhibits a spatio-temporal non-uniformity, which leads to two critical issues that hinder real-world reliability and safety: **Causal Confusion** and the **Planning Horizon Dilemma**.

Causal Confusion. The immense burden of mapping high-dimensional sensor data directly to a trajectory may cause the model to find *shortcuts*, relying on spurious correlations instead of the underlying causal logic governing safe driving [14, 20, 24]. For instance, a model might learn to associate braking with a lead vehicle's brake lights but fail to understand the red light causing that vehicle to stop, leading it to dangerously follow the car through an intersection. **Planning Horizon Dilemma.** Trajectory data becomes more uncertain over longer horizons. Consequently, predictions for these distant waypoints often diverge significantly from the eventual ground truth, resulting in large loss values during training [2, 18]. This skews the optimization process, forcing the model to prioritize large, unpredictable long-term errors over the precision of the critical near-term path essential for immediate collision avoidance.

To address these challenges, we propose ***ResAD***, a Normalized **Res**idual Trajectory Modeling framework for End-to-End **A**utonomous **D**riving. As shown in Fig. 1 **(b)**, our core idea is to decompose the complex prediction task into two distinct components: **(1)** a deterministic physics-based baseline, the **inertial reference**, obtained by extrapolating the vehicle's current state to represent its default trajectory in the absence of active control; and **(2)** a learned **residual**, representing the necessary deviations from the inertial reference. By focusing specifically on deviations rather than the entire trajectory, ***ResAD*** shifts the learning objective from "***What is the future trajectory?***" to "***Why must the trajectory change?***". This shift encourages the model to understand underlying causal factors (*e.g.*, traffic rules, obstacles) instead of exploiting spurious correlations. To further mitigate the adverse effects of spatial scale variations during optimization, we conduct **Point-wise Residual Normalization** on the residuals. This technique prevents high-magnitude residuals at certain trajectory points from dominating the learning signal, ensuring that numerically small yet critically important adjustments are properly captured. Additionally, we strategically perturb the ego-vehicle's state, generating diverse inertial references to counteract planning errors arising from sensor inaccuracies and guide the model toward a broader spectrum of high-quality trajectories. By embedding the fundamental physical prior of inertia into the model's architecture, ***ResAD*** significantly simplifies the learning task, enabling more nuanced and precise driving behaviors. In summary, our contributions are as follows:

- We revisit the future-trajectory-prediction paradigm in E2EAD, and contend that the spatio-temporal non-uniformity of raw trajectory data leads to causal confusion and the planning horizon dilemma. This encourages a paradigm shift, moving from predicting the trajectory itself to modeling the reasons for its deviation.
- We propose ***ResAD***, an E2EAD framework utilizing the **Normalized Residual Trajectory Modeling**. It first obtains an **inertial reference** by extrapolating the vehicle's current state and then learns to predict the **residual**, *i.e.*, the necessary deviations, relative to it. We further apply **Point-wise Residual Normalization** to the residuals, which prevents the optimization process from being dominated by long-horizon uncertainties.
- Extensive experiments and analyses validate the effectiveness of the proposed ***ResAD***. On the NAVSIM benchmark, our method achieves state-of-the-art performance with scores of 88.6 for PDMS and 85.5 for EPDMS.

2. Related Work

2.1. End-to-End Autonomous Driving

End-to-end autonomous driving (E2EAD) seeks to overcome the limitations of traditional modular pipelines, such as error accumulation and inter-module information loss [14, 31, 36]. Pioneering works like UniAD [12] introduced a planning-oriented architecture that jointly optimizes perception and forecasting to mitigate error propagation. VAD [17] further streamlined the pipeline with a fully vectorized scene representation, enabling the enforcement of explicit, instance-level safety constraints. More recently, generative models have become a new frontier in E2EAD research [6, 27, 39]. GoalFlow [34] introduces a goal-conditioned generative model that first selects an optimal goal point based on scene context and then uses Flow Matching to efficiently generate high-quality trajectories towards it. Despite these advances, existing methods predominantly rely on the direct prediction of future trajectories. In this work, we depart from this paradigm and introduce **Normalized Residual Trajectory Modeling**. Our method formulates a trajectory by decomposing it into a physically-based inertial reference and a learnable residual, offering a more structured and interpretable approach to trajectory representation.

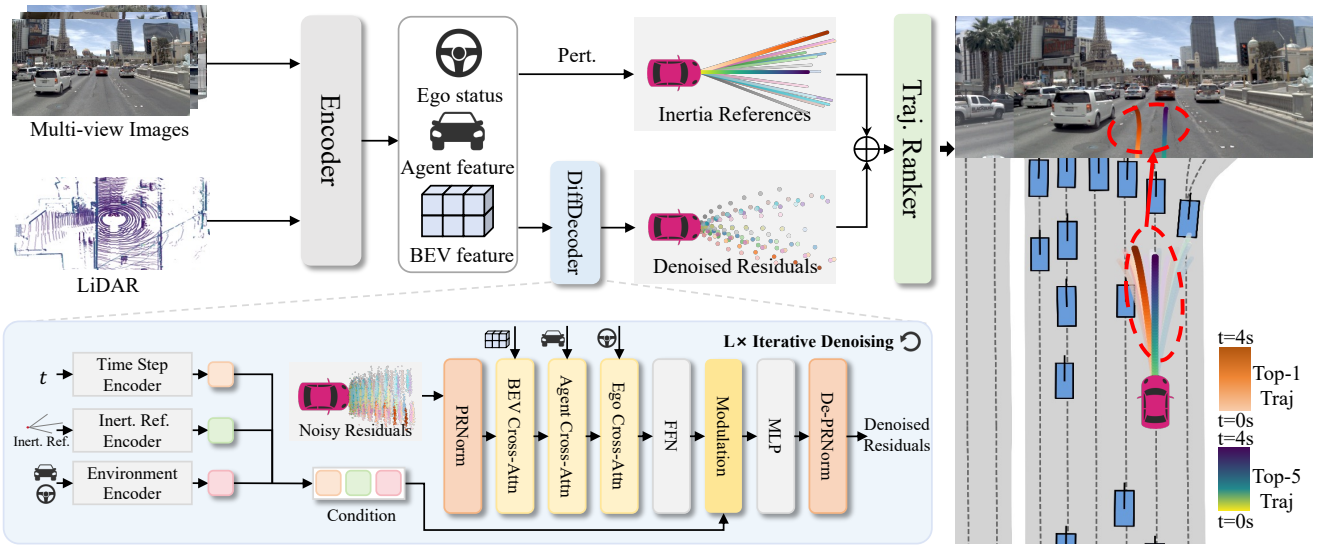

Figure 2. The proposed ***ResAD*** framework. Multi-view images and LiDAR data are first processed and fused by a feature interaction encoder. We generate an inertial reference from the ego-vehicle's state and perturb it into a cluster to ensure robustness and enable multi-modal predictions. Finally, diffusion decoders, conditioned on this reference cluster, merge the encoded features via cross-attention to output the planned trajectories.

2.2. Multimodal Planning

Most E2EAD systems produce a single, deterministic trajectory, an approach that struggles with the inherent diversity of real-world driving scenarios. To address this, several works have explored multimodal planning. VADv2 [3] proposes a probabilistic planning framework that outputs a distribution of future trajectories, which can be sampled to produce a diverse set of behaviors. The Hydra-MDP series [21, 23] employs policy distillation to select multiple trajectories from a vocabulary guided by an expert. GTRS [25] adopts a different strategy, scoring a set of pre-generated trajectories to ensure both diversity and safety. DiffusionDrive [27] highlights the challenge of mode collapse in generative-based models, addressing it by anchoring trajectory generation to a fixed cluster vocabulary. However, these methods fundamentally rely on a static, predefined vocabulary. This makes them both inefficient and restrictive, forcing them to evaluate irrelevant options while being unable to generate truly optimal trajectories outside the discrete set. Differently, ***ResAD*** benefits from the unique trajectory modeling strategy that enables it to directly denoise from the Gaussian noise, yielding superior, context-aware multimodal trajectories.

3. Methodology

3.1. Preliminaries

E2EAD aims to learn a unified policy, π, that directly maps raw sensor inputs, $\mathcal{O}$, to a sequence of waypoints, $\tau = \{(x_t, y_t)\}_{t=1}^{T_f}$, where T_f denotes the planning horizon, and (x_t, y_t) is the predicted future location of each waypoint at time t. We construct the proposed ***ResAD*** using a vanilla diffusion [11, 30] framework. The diffusion model defines a Markovian chain of diffusion forward process q by gradually adding noise to sample data z_0, over a series of T timesteps, which can be formulated as:

$$q(\boldsymbol{z}_t|\boldsymbol{z}_0) = \mathcal{N}(\boldsymbol{z}_t|\sqrt{\bar{\alpha}_t}\boldsymbol{z}_0, (1-\bar{\alpha}_t)\boldsymbol{I}), \tag{1}$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. The hyperparameters β_t controlling the amount of noise added at each step. As $t \to T$, z_T approaches a pure Gaussian noise distribution. We train the denoise model π_θ to predict z_0 from z_i with the guidance of conditional information c, where θ are the trainable parameters. At the inference stage, the trained network is used to iteratively denoise a pure noise $z_T \sim \mathcal{N}(0, \mathbf{I})$ to produce a clean data sample x_0, which is defined as:

$$p_\theta\left(z_0 \mid c\right) = \int p\left(z_T\right) \prod_{i=1}^{T} p_\theta\left(z_{i-1} \mid z_i, c\right) \mathrm{d}z_{1:T}. \tag{2}$$

In this work, we aim to solve the E2EAD via the diffusion model. Instead of directly generating the future trajectory points, we define the data samples as a set of normalized residuals.

3.2. Normalized Residual Trajectory Modeling

As illustrated in Fig. 2, ***ResAD*** takes multi-view images and LiDAR point clouds as input, which are fused by a Transfuser-style encoder. From the ego-vehicle state, we

generate an inertial reference. ***ResAD*** then perturbs this reference into a cluster to ensure robustness to state noise and to enable multi-modal predictions. The Diffusion Decoders employ cross-attention to merge the encoded features, using the inertial reference cluster as a condition to guide the training.

Trajectory Residual Modeling. The core idea of ***ResAD*** is to reframe trajectory prediction as a simpler, more interpretable learning problem. Instead of predicting the entire future trajectory from scratch, the model learns to predict the necessary correction to a simple, physics-based baseline. This baseline is an inertial reference trajectory, extrapolated from the ego-vehicle's current status using a constant velocity model. This reframing compels the model to learn the control interventions required to deviate from the default path, focusing its capacity on the causal elements of driving.

Let the ego-vehicle's velocity be $\boldsymbol{v}_0 = (v_{x,0}, v_{y,0})$ and its position be $\mathbf{p}_0 = (x_0, y_0)$ at the current time $t = 0$. The inertial reference trajectory τ_{ref} for future timesteps t_i in the prediction horizon $T_f = \{t_1, t_2, ..., t_N\}$ is calculated as:

$$\mathbf{p}_{t_i} = \mathbf{p}_0 + \mathbf{v}_0 \cdot \Delta t_i. \tag{3}$$

This reference τ_{ref} represents the path the vehicle would follow with no control inputs. We define the trajectory residual $\boldsymbol{r}$ as the point-wise difference between the ground-truth trajectory τ_{gt} and the reference trajectory τ_{ref}:

$$\boldsymbol{r} = \tau_{\text{gt}} - \tau_{\text{ref}}. \tag{4}$$

This residual $\boldsymbol{r}$quantifies the precise corrections a human driver applied to navigate the environment. The learning objective of ***ResAD*** is thus to predict this residual, effectively capturing the driver's decision-making process. **Point-wise**

Residual Normalization. A key challenge in trajectory prediction is the scale variance of coordinates across the time horizon. Points further in the future have numerically larger values, which can cause the optimization to be dominated by far-field errors, neglecting the fine-grained, safety-critical adjustments required in the near field. As shown in Fig. 1(a), while residual modeling mitigates this by focusing on deviations, the scale issue within the residuals themselves persists. We propose Point-wise Residual Normalization (PRNorm) to resolve this.

Given a residual trajectory $\boldsymbol{r}$, which is a sequence of T_f displacement vectors $\{r_1, r_2, \ldots, r_{T_f}\}$, where each $r_t = (r_t^x, r_t^y)$ is a 2D vector. A standard min-max scaling is performed on a component-wise basis for each dimension $d \in \{x, y\}$. The extremal values, $r^d_{\min}$ and $r^d_{\max}$ are precomputed across all timesteps and all trajectories in the entire training dataset:

$$r^d_{\min} = \min_{j,t}(r^d_{j,t}), \quad r^d_{\max} = \max_{j,t}(r^d_{j,t}), \tag{5}$$

where j indexes trajectories in the training set and t indexes the timestep. These values define the tightest axis-aligned bounding box for the residual. To provide fine-grained control over the final feature distribution, we introduce a hyperparameter $\gamma > 0$. This parameter defines the bounds of the symmetric output interval $[-\gamma, \gamma]$. The complete transformation of PRNorm for each component r^d_t of every vector r_t is given by:

$$\tilde{r}^d_t = 2\gamma \left(\frac{r^d_t - r^d_{\min}}{r^d_{\max} - r^d_{\min} + \epsilon_0} \right) - \gamma. \tag{6}$$

The small constant ϵ_0 is added to the denominator to ensure numerical stability. Through this, we can get the normalized residual $\tilde{\boldsymbol{r}} = \text{PRNorm}(\boldsymbol{r})$.

Inertia Reference Perturbation. Driving is an inherently multi-modal task. Most methods depend on a fixed trajectory vocabulary, where most options are irrelevant to the current scene, causing inefficiency. ***ResAD***, circumvents this issue through Trajectory Residual Modeling, which generates multi-modal trajectories by perturbing its Inertial Reference. This approach is doubly beneficial. It forces the model to learn resilience against noise from ego-sensors like GPS and IMU. On the other hand, it creates a set of intent hypotheses by generating a cluster of slightly varied inertial references. The network then produces a full trajectory for each hypothesis, naturally yielding a diverse set of context-relevant paths.

Specifically, we introduce stochastic perturbations directly into the initial velocity $\boldsymbol{v}_0$. We generate K distinct perturbation vectors $\boldsymbol{\delta}_{\mathbf{v},k}$ by sampling from a zero-mean multivariate Gaussian distribution:

$$\boldsymbol{\delta}_{\mathbf{v},k} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}) \quad \text{for} \quad k = 1, \ldots, K. \tag{7}$$

Here, the covariance matrix $\boldsymbol{\Sigma} = \text{diag}(\sigma^2_{vx}, \sigma^2_{vy})$ governs the variance of the perturbations along the longitudinal and lateral axes, respectively. These hyperparameters effectively define the exploration scope of our model's initial hypotheses. Each perturbation is additively fused with the original velocity vector to forge K unique, perturbed initial states. By propagating each of these perturbed velocity vectors $\mathbf{v}'_{0,k}$ through the constant velocity model Eq. 3, we generate a set of K distinct inertial references $\{\tau_{\text{ref},\,k}\}_{k=1}^K$ and corresponding residuals:

$$\begin{aligned} \mathbf{v}'_{0,k} &= \mathbf{v}_0 + \delta_{\mathbf{v},k}, \\ \{\boldsymbol{r}_k\}_{k=1}^K &= \{\tau_{\text{gt}} - \tau_{\text{ref},\,k}\}_{k=1}^K. \end{aligned} \tag{8}$$

Training and Inference. In training, adding Gaussian noise to the residual cluster normalized by PRNorm:

$$\boldsymbol{z}_k^{(i)} = \sqrt{\bar{\alpha}_i}\tilde{\boldsymbol{r}}_k + \sqrt{1 - \bar{\alpha}_i}\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}), \tag{9}$$

Table 1. **Performance on the NAVSIM v1 NAVTEST Benchmark.**

Method	Input	Backbone	NC ↑	DAC ↑	EP ↑	TTC ↑	C ↑	PDMS ↑
Transfuser [7]	C & L	Resnet-34	97.7	92.8	79.2	92.8	**100**	84.0
UniAD [12]	C	Resnet-34	97.8	91.9	78.8	92.9	**100**	83.4
VADv2 [3]	C & L	Resnet-34	97.2	89.1	76.0	91.6	**100**	80.9
PARA-Drive [33]	C	Resnet-34	97.9	92.4	79.3	93.0	99.8	84.0
DRAMA [38]	C & L	Resnet-34	98.0	93.1	80.1	<u>94.8</u>	**100**	85.5
Hydra-MDP* [23]	C & L	Resnet-34	<u>98.3</u>	96.0	78.7	94.6	**100**	86.5
Hydra-MDP++* [21]	C	Resnet-34	97.6	96.0	80.4	93.1	**100**	86.6
GoalFlow* [34]	C & L	Resnet-34	<u>98.3</u>	93.8	79.8	94.3	**100**	85.7
ARTEMIS [9]	C & L	Resnet-34	<u>98.3</u>	95.1	81.4	94.3	**100**	87.0
DiffusionDrive [27]	C & L	Resnet-34	98.2	96.2	<u>82.2</u>	94.7	**100**	88.1
WoTE [22]	C & L	Resnet-34	**98.5**	<u>96.8</u>	81.9	**94.9**	<u>99.9</u>	<u>88.3</u>
ResAD	C & L	Resnet-34	98.0	**97.3**	**82.5**	94.2	**100**	**88.6**
GoalFlow* [34]	C & L	V2-99	<u>98.4</u>	**98.3**	85.0	<u>94.6</u>	**100**	90.3
Hydra-MDP* [23]	C & L	V2-99	98.0	<u>97.8</u>	<u>86.5</u>	93.9	**100**	90.3
ResAD	C & L	V2-99	**98.9**	<u>97.8</u>	**87.0**	**94.9**	**100**	**90.6**

"C": Camera, "L": LiDAR. The best and second-best scores are highlighted in **bold** and <u>underlined</u>, respectively. * For fair comparison, we use the official scores of versions with the same backbone.

where $\tilde{\boldsymbol{r}}_k = \mathrm{PRNorm}(\boldsymbol{r}_k)$. The diffusion decoder f_θ takes K noisy normed trajectory residuals to generate denoised residuals $\{\hat{\boldsymbol{r}}_k\}_{k=1}^K$:

$$\{\hat{\boldsymbol{r}}_k\}_{k=1}^K = f_\theta(\{\boldsymbol{z}_k^{(i)}\}_{k=1}^K, c), \tag{10}$$

where c represents the conditional information. Note that c is composed of query features extracted from the encoder and the corresponding timestep embedding. Crucially, c also incorporates unique positional encoding features derived from each of the perturbed inertial references. These encodings are essential for the model to distinguish between the different intent hypotheses and subsequently generate a diverse set of trajectories. The diffusion loss is computed as:

$$\mathcal{L}_{\text{diff}} = \sum_{k=1}^{K} \mathcal{L}_{\text{rec}}(\hat{\boldsymbol{r}}_k, \boldsymbol{r}_k), \tag{11}$$

here, $\mathcal{L}_{\text{rec}}$ can be a simple L1 loss or MSE loss.

During inference, the denoising process starts with K_{infer} Gaussian noise to generate residuals. We take 2 timesteps to get the final predictions $\{\hat{\boldsymbol{r}}_k\}_{k=1}^{K_{\text{infer}}}$ by DDIM [30]. Then the predicted residuals are added to the corresponding perturbed inertia reference to get the multi-modal trajectory $\{\hat{\tau}_k\}_{k=1}^{K_{\text{infer}}}$.

3.3. Multimodal Trajectory Ranker

Inspired by VADv2 [3] and Hydra-MDP [23], we develop a Trajectory Ranker to select the optimal trajectory from multiple modalities by using the output from the planning model. Given a set of trajectory candidates v_k, where k is the vocabulary size, we feed them into a Transformer to facilitate interaction with the perception representations, E_{env}, which can be expressed as follows:

$$\begin{aligned} \mathcal{V} &= \mathrm{PosEmb}(v_k), \\ \mathcal{V}' &= \mathrm{Transformer}(Q = \mathcal{V}, K, V = E_{env}) + E. \end{aligned} \tag{12}$$

$\mathrm{PosEmb}(\cdot)$ denotes the position embedding, and ego status E is embedded into the transformer output. Subsequently, the latent vector $\mathcal{V}'$ is fed into a set of MLP heads to predict the score $\{\hat{\mathcal{S}}_i^m | i = 1, ..., k\}_{m=1}^{|M|}$ for each metric $m \in M$ and the i-th trajectory, where M represents the set of metrics used in PDMS or EPDMS. The ranker is trained with the ground truth score $\{\mathcal{S}_i^m | i = 1, ..., k\}_{m=1}^{|M|}$ to distill the knowledge from the rule-based planner and the ground truth waypoints as follows:

$$\mathcal{L}_{\text{ranker}} = \sum_{i=1}^{k} y_i \log(\hat{\mathcal{S}}_i^{im}) + \sum_{m,i} \mathrm{BCE}(\mathcal{S}_i^m, \hat{\mathcal{S}}_i^m), \tag{13}$$

where $y_i = \frac{e^{-(\tau_{\text{gt}} - \hat{\tau}_i)^2}}{\sum_{j=1}^{k} e^{-(\tau_{\text{gt}} - \hat{\tau}_j)^2}}$. During inference, we compute scores for the outputs of the planning head and select the trajectory with the highest weighted score as the final output.

4. Experiments

4.1. Benchmark

We evaluate the proposed ***ResAD*** on the NAVSIM v1 [8] and NAVSIM v2 [1] benchmark. NAVSIM is built upon

Table 2. **Performance on the NAVSIM v2 NAVTEST Benchmark with Extended Metrics.**

Method	NC ↑	DAC ↑	DDC ↑	TL ↑	EP ↑	TTC ↑	LK ↑	HC ↑	EC ↑	EPDMS ↑
Ego Status MLP	93.1	77.9	92.7	99.6	86.0	91.5	89.4	98.3	85.4	64.0
Transfuser [7]	96.9	89.9	97.8	99.7	87.1	95.4	92.7	98.3	87.2	76.7
HydraMDP++ [21]	97.2	**97.5**	99.4	99.6	83.1	96.5	94.4	98.2	70.9	81.4
DriveSuprim [36]	97.5	96.5	99.4	99.6	**88.4**	96.6	95.5	98.3	77.0	83.1
ARTEMIS [9]	**98.3**	95.1	98.6	**99.8**	81.5	**97.4**	96.5	98.3	-	83.1
DiffusionDrive [27]	98.2	95.9	99.4	**99.8**	87.5	97.3	96.8	98.3	87.7	84.5
ResAD	97.8	97.2	**99.5**	**99.8**	88.2	96.9	**97.0**	**98.4**	**88.2**	**85.5**

the real-world NuPlan dataset [19] and exclusively features relevant annotations and sensor data sampled at 2 Hz. The NAVSIM dataset contains two parts: NAVTRAIN and NAVTEST, including 1192 and 136 scenarios respectively, used for trainval and test.

NAVSIM v1. In this benchmark, each predicted trajectory is sent to a simulator, which validates the driving metrics in the corresponding environment. The planning capabilities of models are assessed using the PDM score (PDMS), which is calculated as follows:

$$\text{PDMS} = \text{NC} \times \text{DAC} \times \frac{(5 \times \text{TTC} + 2 \times \text{C} + 5 \times \text{EP})}{12}, \tag{14}$$

where the sub-metrics NC, DAC, TTC, C, EP represent the No At-Fault Collisions, Drivable Area Compliance, Time to Collision, Comfort, and Ego Progress.

NAVSIM v2. In NAVSIM v2, a new Extended PDM Score (EPDMS) is introduced in NAVSIM v2 , which can be formulated as:

$$\begin{aligned}\text{EPDMS} = \text{NC} \times \text{DAC} \times \text{DDC} \times \text{TL} \times \\ \frac{(5 \times \text{TTC} + 2 \times \text{C} + 5 \times \text{EP} + 5 \times \text{LK} + 5 \times \text{EC})}{22}.\end{aligned} \tag{15}$$

The extended sub-metrics DDC, TL, LK, and EC correspond to the Driving Direction Compliance, Traffic Lights Compliance, Lane Keeping Ability, and Extended Comfort.

4.2. Implementation Details

For fair comparison, our model adopts an identical perception module and ResNet-34 backbone as Transfuser [7]. The model takes two types of input: three forward-facing camera images, which are individually cropped, downscaled, and then concatenated into a single 1024×256 tensor; and a rasterized BEV representation of the LiDAR point cloud. ***ResAD*** is equipped with 2 cascaded diffusion layers. We set the mode number $K_{\text{train}} = 20$ for training and $K_{\text{infer}} = 200$ for testing. The model is trained from scratch on the NAVTRAIN split for 100 epochs using the DDPM, with a timestep T of 1000. The training is distributed across 8 NVIDIA L20 GPUs, with a total batch size of 512, and is optimized using AdamW. The ranker's training leverages a fixed trajectory vocabulary and the output from our frozen, pre-trained diffusion model to learn a scoring function. In inference, we use DDIM to sample the predictions with only 2 denoising steps. The resulting candidates are then evaluated by the trained ranker, which selects the highest-scoring trajectory as the output. We predict $T_f = 8$ timesteps and the interval between each time step is 0.5s. For more details, please refer to the supplementary material.

4.3. Main Results

Quantitative Comparison. The results presented in Tab. 1 show that ***ResAD*** achieves a state-of-the-art performance on NAVSIM v1 navtest split, with a PDMS of 88.6. Our NC of 98.0 is on par with the highest scores, ensuring a high level of safety by minimizing collisions. The EP of 82.5 achieved by our model is a notable result, indicating efficient route completion. ***ResAD*** excels in DAC with a score of 97.3, outperforming WoTE's 96.8. This suggests our model has a stronger adherence to lane boundaries and drivable areas, a critical aspect of safe and predictable driving behavior. On the more challenging NAVSIM v2 benchmark, the advantages of ***ResAD*** are further extended. As shown in Tab. 2, ***ResAD*** achieves the best or second-best performance across almost all extended sub-metrics. Specifically, ***ResAD*** achieves an EPDMS of 85.5, surpassing DiffusionDrive by 1.0. It achieves a higher EP score of 88.2 (vs. 87.5), indicating it completes routes more effectively. Furthermore, it shows a significant advantage in DAC with a score of 97.2 versus 95.9, confirming its ability to generate more precise trajectories that better adhere to lane boundaries. ***ResAD*** also exhibits finer vehicle handling, with slightly better scores in LK.

Qualitative Comparison. A qualitative comparison on NAVSIM (Fig. 3) highlights the different multimodal strategies of ***ResAD*** and DiffusionDrive. While both success-

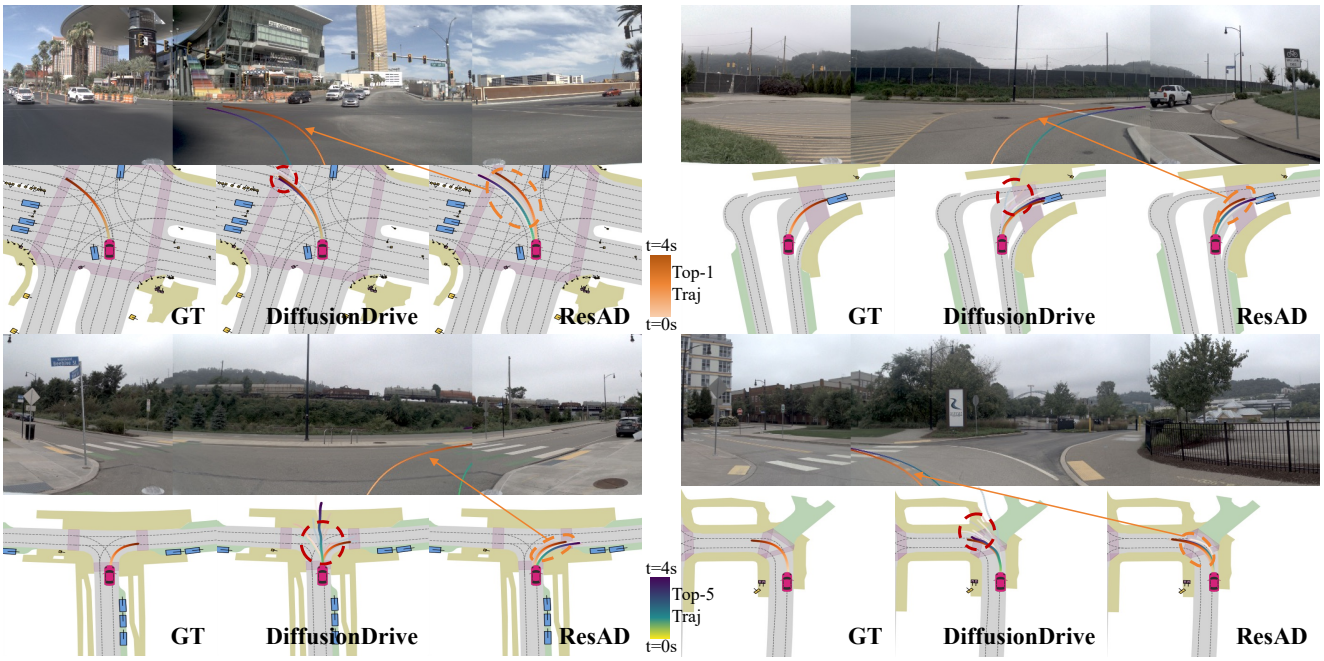

Figure 3. **Visual comparison of *ResAD***. DiffusionDrive relies on a static, context-agnostic vocabulary, often proposing infeasible trajectories (circled in red). In contrast, the proposed ***ResAD*** dynamically generates context-aware trajectories by perturbing the ego-vehicle's velocity, addressing limitations in static approaches.

fully avoid the mode collapse typical of vanilla diffusion, their underlying approaches diverge significantly. DiffusionDrive relies on a static, predefined trajectory vocabulary. This context-agnostic approach forces it to generate many irrelevant or unfeasible options, such as proceeding straight in a sharp turn scenario (**highlighted by red circles in the figure**). Although a subsequent filtering step can prune these invalid paths, this two-stage process is inherently inefficient. In contrast, ***ResAD*** overcomes this limitation through the distinct trajectory modeling shift. This is achieved through a mechanism of perturbing the ego-vehicle's velocity. It directly explores a set of plausible behaviors, generating trajectories that are inherently consistent with the immediate driving context. A **real-world vehicle demonstration** of the proposed method is available in the supplementary material.

Table 3. **Ablation study on the influence of each component.**

Model	Description	NC ↑	DAC ↑	EP ↑	TTC ↑	C ↑	PDMS ↑
$\mathcal{M}_0$	Base Model	97.8	94.2	78.1	93.4	100	84.9
$\mathcal{M}_1$	$\mathcal{M}_0$ + Ranker	98.3	94.3	77.8	94.6	100	85.1
$\mathcal{M}_2$	$\mathcal{M}_1$ + TRM	97.4	96.6	80.3	93.2	100	86.3
$\mathcal{M}_3$	$\mathcal{M}_2$ + PRNorm	97.6	96.7	81.4	93.3	100	87.2
$\mathcal{M}_4$	$\mathcal{M}_3$ + IRP	98.0	97.3	82.5	94.2	100	88.6

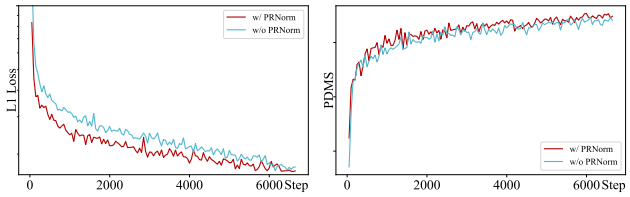

Figure 4. **The impact of PRNorm on training efficiency and performance.** This figure shows the loss and mean PDMS curves for ***ResAD*** trained with or without the proposed PRNorm.

4.4. Ablation Studies

Component Analysis. To validate the efficacy of each proposed component in ***ResAD***, we conduct a comprehensive ablation study, with the results detailed in Table 3. Our analysis begins by integrating the Multimodal Trajectory Ranker module into the base model, forming $\mathcal{M}_1$. It offers only a slight performance benefit, suggesting poor multimodal ability as its outputs were mostly limited to a small area. The introduction of Trajectory Residual Modeling (TRM) significantly boosted performance, with the DAC metric improving from 94.3 to 96.6 and EP from 77.8 to 80.3, underscoring its role in improving path completion and safety. In addition, integrating PRNorm enhanced performance, particularly for EP, which demonstrates its value in normalizing feature representations and accelerat-

Table 4. **Extending Normalized Residual Trajectory Modeling to other models.**

Model	NC ↑	DAC ↑	EP ↑	TTC ↑	C ↑	PDMS ↑
Transfuser	97.7	92.8	79.2	92.8	100	84.0
+TRM	97.7	93.5	80.0	93.5	100	85.2
+PRNorm	98.0	94.2	79.8	93.6	99.9	85.6
$\text{Transfuser}_{\text{DP}}$	97.4	93.5	79.0	93.0	100	84.5
+TRM	98.0	93.9	80.2	93.6	100	85.5
+PRNorm	98.2	94.8	79.4	94.2	100	85.8

ing training. Finally, incorporating Inertia Reference Perturbation (IRP) to enhance multi-modal planning brought substantial gains, increasing the PDMS score from 87.2 to 88.6. The remarkable effectiveness of IRP is enabled by our Normalized Residual Trajectory Modeling approach. By deconstructing the trajectory data, our model can cleverly foster multi-modality without relying on a fixed trajectory vocabulary, allowing ***ResAD*** to generate a diverse set of trajectories that are better aligned with the current environmental state.

As shown in Fig. 4, PRNorm enables a significantly faster decline in loss compared to the baseline (vanilla min-max normalization), accelerating model convergence. Furthermore, we calculate the PDMS of the predicted trajectory every step, which is also higher with PRNorm. It demonstrates its comprehensive benefits to both training efficiency and the final performance.

Effect of the Normalized Residual Trajectory Modeling. To further validate the effectiveness of our proposed Normalized Residual Trajectory Modeling, we conducted extensive experiments on two heterogeneous planning models. The results are shown in Tab. 4. Specifically, we evaluated it on Transfuser, which represents the MLP-based planning network, and $\text{Transfuser}_{\text{DP}}$, which is an extension of Transfuser incorporating a UNet diffusion decoder, representing the diffusion-based planning network. Our findings consistently demonstrate that the proposed Normalized Residual Trajectory Modeling significantly enhances trajectory quality across both types of planning networks. The integration of the proposed TRM and PRNorm yields notable improvements across several crucial performance metrics. For the Transfuser baseline, the PDMS is improved from 84.0 to 85.4 with the help of TRM. With the engagement of PRNorm, the PDMS is further increased to 85.9. Consistent improvements are also observed with $\text{Transfuser}_{\text{DP}}$. Consistent gains on diverse metrics validate Normalized Residual Trajectory Modeling as a generalizable and effective method for improving the safety and reliability of E2EAD systems.

5. Conclusion

In this work, we revisit the conventional future-trajectory-prediction paradigm in E2EAD. We argue that directly predicting a vehicle's trajectory from sensor data forces models into a state of causal confusion and creates a planning horizon dilemma, undermining safety and reliability. Our proposed framework, ***ResAD***, confronts these challenges by reframing the learning objective. By first establishing a deterministic **inertial reference**, we provide a strong physical prior that anchors the prediction task. The model then learns to predict the **residual**, *i.e.*, the necessary deviations from this baseline, which encourages it to focus on the external causal factors, such as obstacles and traffic rules, that govern safe navigation. Furthermore, we introduced Point-wise Residual Normalization (PRNorm) to specifically tackle the optimization imbalance. PRNorm re-weights the learning objective at each waypoint, preventing large-magnitude errors from distant, uncertain predictions from dominating the training process and ensuring that critical, near-term adjustments are properly prioritized. Our state-of-the-art results on NAVSIM demonstrate that this conceptual shift significantly simplifies the learning task and provides a more robust foundation for future E2EAD systems.

References

[1] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, Xunjiang Gu, Caojun Wang, Yakov Miron, Marco Aiello, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Pseudo-simulation for autonomous driving. *arXiv*, 2506.04218, 2025. 5

[2] Li Chen, Penghao Wu, Kashyap Chitta, Bernhard Jaeger, Andreas Geiger, and Hongyang Li. End-to-end autonomous driving: Challenges and frontiers. *IEEE Trans. Pattern Anal. Mach. Intell.*, 2024. 1, 2

[3] Shaoyu Chen, Bo Jiang, Hao Gao, Bencheng Liao, Qing Xu, Qian Zhang, Chang Huang, Wenyu Liu, and Xinggang Wang. Vadv2: End-to-end vectorized autonomous driving via probabilistic planning. *arXiv preprint arXiv:2402.13243*, 2024. 2, 3, 5

[4] Zhili Chen, Maosheng Ye, Shuangjie Xu, Tongyi Cao, and Qifeng Chen. Ppad: Iterative interactions of prediction and planning for end-to-end autonomous driving. In *Eur. Conf. Comput. Vis.*, pages 239–256, 2024. 2

[5] Zesong Chen, Ze Yu, Jun Li, Linlin You, and Xiaojun Tan. Dualat: Dual attention transformer for end-to-end autonomous driving. In *Proc. of the IEEE Intl. Conf. on Robot. and Autom.*, pages 16353–16359. IEEE, 2024. 2

[6] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *Proc. of Robot.: Sci. and Syst.*, 2023. 2

[7] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. Transfuser: Imitation

with transformer-based sensor fusion for autonomous driving. *IEEE Trans. Pattern Anal. Mach. Intell.*, 45(11):12878–12895, 2022. 1, 5, 6

[8] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, et al. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. *Adv. Neural Inform. Process. Syst.*, 37: 28706–28719, 2024. 5

[9] Renju Feng, Ning Xi, Duanfeng Chu, Rukang Wang, Zejian Deng, Anzheng Wang, Liping Lu, Jinxiang Wang, and Yanjun Huang. Artemis: Autoregressive end-to-end trajectory planning with mixture of experts for autonomous driving. *arXiv preprint arXiv:2504.19580*, 2025. 5, 6

[10] Hao Gao, Shaoyu Chen, Bo Jiang, Bencheng Liao, Yiang Shi, Xiaoyang Guo, Yuechuan Pu, Haoran Yin, Xiangyu Li, Xinbang Zhang, et al. Rad: Training an end-to-end driving policy via large-scale 3dgs-based reinforcement learning. *arXiv preprint arXiv:2502.13144*, 2025. 2

[11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Adv. Neural Inform. Process. Syst.*, 33:6840–6851, 2020. 3

[12] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, et al. Planning-oriented autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 17853–17862, 2023. 1, 2, 5

[13] Bernhard Jaeger, Kashyap Chitta, and Andreas Geiger. Hidden biases of end-to-end driving models. In *Int. Conf. Comput. Vis.*, 2023. 2

[14] Xiaosong Jia, Yulu Gao, Li Chen, Junchi Yan, Patrick Langechuan Liu, and Hongyang Li. Driveadapter: Breaking the coupling barrier of perception and planning in end-to-end autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7953–7963, 2023. 2

[15] Xiaosong Jia, Penghao Wu, Li Chen, Jiangwei Xie, Conghui He, Junchi Yan, and Hongyang Li. Think twice before driving: Towards scalable decoders for end-to-end autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, 2023. 2

[16] Xiaosong Jia, Junqi You, Zhiyuan Zhang, and Junchi Yan. Drivetransformer: Unified transformer for scalable end-to-end autonomous driving. In *Int. Conf. Learn. Represent.*, 2025. 2

[17] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. Vad: Vectorized scene representation for efficient autonomous driving. In *Int. Conf. Comput. Vis.*, pages 8340–8350, 2023. 2

[18] Sicong Jiang, Zilin Huang, Kangan Qian, Ziang Luo, Tianze Zhu, Yang Zhong, Yihong Tang, Menglin Kong, Yunlong Wang, Siwen Jiao, et al. A survey on vision-language-action models for autonomous driving. *arXiv preprint arXiv:2506.24044*, 2025. 2

[19] Napat Karnchanachari, Dimitris Geromichalos, Kok Seang Tan, Nanxiang Li, Christopher Eriksen, Shakiba Yaghoubi, Noushin Mehdipour, Gianmarco Bernasconi, Whye Kit Fong, Yiluan Guo, et al. Towards learning-based planning: The nuplan benchmark for real-world autonomous driving. In *Proc. of the IEEE Intl. Conf. on Robot. and Autom.*, pages 629–636. IEEE, 2024. 6

[20] Jiankun Li, Hao Li, Jiangjiang Liu, Zhikang Zou, Xiaoqing Ye, Fan Wang, Jizhou Huang, Hua Wu, and Haifeng Wang. Exploring the causality of end-to-end autonomous driving. *arXiv preprint arXiv:2407.06546*, 2024. 2

[21] Kailin Li, Zhenxin Li, Shiyi Lan, Yuan Xie, Zhizhong Zhang, Jiayi Liu, Zuxuan Wu, Zhiding Yu, and Jose M Alvarez. Hydra-mdp++: Advancing end-to-end driving via expert-guided hydra-distillation. *arXiv preprint arXiv:2503.12820*, 2025. 3, 5, 6

[22] Yingyan Li, Yuqi Wang, Yang Liu, Jiawei He, Lue Fan, and Zhaoxiang Zhang. End-to-end driving with online trajectory evaluation via bev world model. In *Int. Conf. Comput. Vis.*, 2025. 5

[23] Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, Yishen Ji, Zhiqi Li, Ziyue Zhu, Jan Kautz, Zuxuan Wu, et al. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024. 3, 5

[24] Zhiqi Li, Zhiding Yu, Shiyi Lan, Jiahan Li, Jan Kautz, Tong Lu, and Jose M Alvarez. Is ego status all you need for open-loop end-to-end autonomous driving? In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 14864–14873, 2024. 2

[25] Zhenxin Li, Wenhao Yao, Zi Wang, Xinglong Sun, Joshua Chen, Nadine Chang, Maying Shen, Zuxuan Wu, Shiyi Lan, and Jose M. Alvarez. Generalized trajectory scoring for end-to-end multimodal planning, 2025. 3

[26] Ming Liang, Bin Yang, Wenyuan Zeng, Yun Chen, Rui Hu, Sergio Casas, and Raquel Urtasun. Pnpnet: End-to-end perception and prediction with tracking in the loop. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 11553–11562, 2020. 1

[27] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, et al. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 12037–12047, 2025. 2, 3, 5, 6

[28] Shuai Liu, Quanmin Liang, Zefeng Li, Boyang Li, and Kai Huang. Gaussianfusion: Gaussian-based multi-sensor fusion for end-to-end autonomous driving. *arXiv preprint arXiv:2506.00034*, 2025. 2

[29] Wenjie Luo, Bin Yang, and Raquel Urtasun. Fast and furious: Real time end-to-end 3d detection, tracking and motion forecasting with a single convolutional net. In *Int. Conf. Comput. Vis.*, pages 3569–3577, 2018. 1

[30] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *Int. Conf. Learn. Represent.*, 2021. 3, 5

[31] Ziying Song, Caiyan Jia, Lin Liu, Hongyu Pan, Yongchang Zhang, Junming Wang, Xingyu Zhang, Shaoqing Xu, Lei Yang, and Yadan Luo. Don't shake the wheel: Momentum-aware planning in end-to-end autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 22432–22441, 2025. 2

[32] Wenchao Sun, Xuewu Lin, Yining Shi, Chuang Zhang, Haoran Wu, and Sifa Zheng. Sparsedrive: End-to-end autonomous driving via sparse scene representation. *Proc. of the IEEE Intl. Conf. on Robot. and Autom.*, 2025. 2

[33] Xinshuo Weng, Boris Ivanovic, Yan Wang, Yue Wang, and Marco Pavone. Para-drive: Parallelized architecture for real-time autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 15449–15458, 2024. 2, 5

[34] Zebin Xing, Xingyu Zhang, Yang Hu, Bo Jiang, Tong He, Qian Zhang, Xiaoxiao Long, and Wei Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 1602–1611, 2025. 2, 5

[35] Zetong Yang, Li Chen, Yanan Sun, and Hongyang Li. Visual point cloud forecasting enables scalable autonomous driving. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 14673–14684, 2024. 2

[36] Wenhao Yao, Zhenxin Li, Shiyi Lan, Zi Wang, Xinglong Sun, Jose M Alvarez, and Zuxuan Wu. Drivesuprim: Towards precise trajectory selection for end-to-end planning. *arXiv preprint arXiv:2506.06659*, 2025. 2, 6

[37] Tengju Ye, Wei Jing, Chunyong Hu, Shikun Huang, Lingping Gao, Fangzhen Li, Jingke Wang, Ke Guo, Wencong Xiao, Weibo Mao, et al. Fusionad: Multi-modality fusion for prediction and planning tasks of autonomous driving. *arXiv preprint arXiv:2308.01006*, 2023. 2

[38] Chengran Yuan, Zhanqi Zhang, Jiawei Sun, Shuo Sun, Zefan Huang, Christina Dao Wen Lee, Dongen Li, Yuhang Han, Anthony Wong, Keng Peng Tee, et al. Drama: An efficient end-to-end motion planner for autonomous driving with mamba. *arXiv preprint arXiv:2408.03601*, 2024. 5

[39] Wenzhao Zheng, Ruiqi Song, Xianda Guo, Chenming Zhang, and Long Chen. Genad: Generative end-to-end autonomous driving. In *Eur. Conf. Comput. Vis.*, pages 87–104, 2024. 2